

Manuscript Number: COMLAN-D-08-00005

Title: A Framework for Attaching Personal Assistants to Existing Applications

Article Type: Full Length Article

Section/Category:

Keywords: Personal Assistants; Interface Agents; Object Oriented Frameworks

Corresponding Author: Sr. Marcelo Gabriel Armentano, Mg.

Corresponding Author's Institution:

First Author: Marcelo G Armentano, Mg

Order of Authors: Marcelo Gabriel Armentano, Mg.; Marcelo G Armentano, Mg; Analía A Amandi, Dr

Manuscript Region of Origin:

Abstract: Personal assistants are computer programs that enhance the functionality provided by a software application by implementing a mixed initiative interaction with the user in which both, the assistant and the user, can execute tasks and initiate the interaction with each other. This kind of software is also known as Interface Agents in the Artificial Intelligence community. A key problem in systems implemented so far is that the interface agent and the application are seen as a unit of software and the code of the agent is embedded in the code of the application resulting in highly coupled and hard to maintain systems. In this paper we present a solution to this problem. We propose a framework that will allow attaching an interface agent to a conventional application without modifying the application in any way. This will allow us to enhance an existent application with an interface agent to assist the user for which we do not have the source code available.

A FRAMEWORK FOR ATTACHING PERSONAL ASSISTANTS TO EXISTING APPLICATIONS

Marcelo G. ARMENTANO and Analía A. AMANDI

ISISTAN Research Institute, UNICEN, Campus Universitario, CP 7000, Tandil, Bs. As.,

Argentina. Tel +54 (2293) 439682 Fax: +54 (2293) 439681

Also CONICET Consejo Nacional de Investigaciones Científicas y Técnicas, Buenos
Aires, Argentina.

e-mail: {marmenta,amandi}@exa.unicen.edu.ar

Abstract. Personal assistants are computer programs that enhance the functionality provided by a software application by implementing a mixed initiative interaction with the user in which both, the assistant and the user, can execute tasks and initiate the interaction with each other. This kind of software is also known as Interface Agents in the Artificial Intelligence community. A key problem in systems implemented so far is that the interface agent and the application are seen as a unit of software and the code of the agent is embedded in the code of the application resulting in highly coupled and hard to maintain systems. In this paper we present a solution to this problem. We propose a framework that will allow attaching an interface agent to a conventional application without modifying the application in any way. This will allow us to enhance an existent application with an interface agent to assist the user for which we do not have the source code available.

Keywords— Personal Assistants, Interface Agents, Object Oriented Frameworks

1 Introduction

The currently dominant interaction metaphor between humans and computers is direct manipulation. In this style of interaction the user is responsible of initiating all tasks

explicitly and to monitor all events. But the more users expect from software applications, the more complex these became. So, a new kind of interaction is needed.

Techniques from the field of Artificial Intelligence (AI) and Human-Computer Interaction (HCI), in particular so-called interface agents (1), can be used to implement a complementary style of interaction, which has been referred to as indirect management. Instead of user-initiated interaction via commands and/or direct manipulation, the user is engaged in a cooperative process in which human and computer agents both initiate communication, monitor events and perform tasks. Moreover, these agents can show a mixed-initiative interaction with their users, detecting when they could help their users and either suggest an action or directly act for helping in the detected situation.

Interface agents are computer programs that employ a variety of techniques to provide active assistance to a user with computer-based tasks. This approach follows the metaphor of a personal assistant who is collaborating with the user in the work environment. The assistant becomes progressively more helpful, as it learns the user's interests, habits and preferences by observing his¹ interaction with the application and will act according to such information.

Interface agents have the ability of receiving orders from human users and answering in a personalized way. But agents not only obey users but also can take the initiative and offer assistance, or directly act on behalf of the user. Figure 1 shows the interface agent approach.

¹ For simplicity, we use “it” for the agent and “he” for the user, but we do not mean any distinctions about sexes.

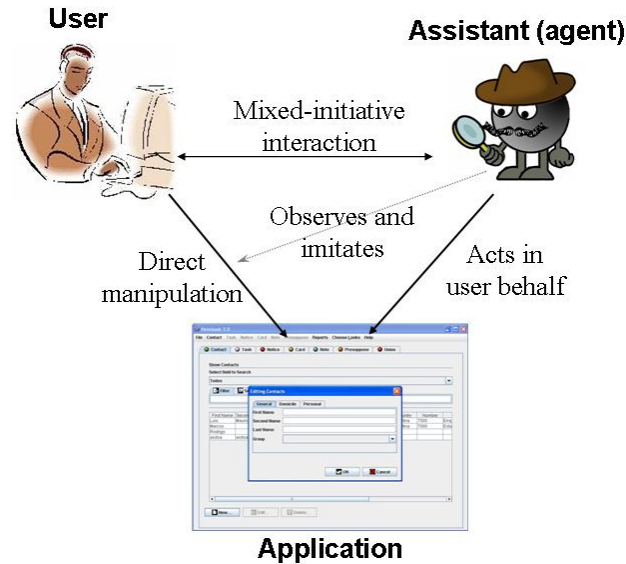


Figure 1: The interface agents' paradigm

The user controls the application via direct manipulation. On the other hand, the agent can act on the application on behalf of the user. The interaction between the user and the agent is given by a mixed-initiative approach. Moreover, the agent observes the user interaction with the application to learn about his habits and preferences with the purpose of adapting the interaction to his particular needs.

A software application can be viewed as a set of tools. Each of these tools aims at helping its users to perform a specific task in the domain of the application. It is very important for an interface agent to know in every moment the task the user is performing because it gives a context in which the user is moving through. Taking this context into account, the agent may infer the user's intention and try to collaborate with the user. Moreover, the context will allow the agent to answer to the user's requirements always in the realm of the current task. For example, if the agent observes that the user is scheduling a work meeting for the following day, the agent can offer to automatically complete the information required and to send an email to each participant of the meeting, providing that

it knows the user's preferences about the kind of meeting he is scheduling.

Interface agents implemented so far are embedded in the application they are designed for. The agent code is usually interwoven with the application code resulting in designs that are difficult to maintain and modify. Furthermore, existent applications for which we do not have their code available, or we cannot modify in any way can be enhanced with the functionality provided by an interface agent, even when these applications are not prepared to these improvements. For these reasons, a separate development of both unit of software is desirable. The interface agent should observe the tasks the user performs on the software application in a non intrusive way.

A key advantage of building interface agents that work on top of existing applications instead of rewriting some part of the application to support the agent's needs of observation is that both units of software will be able to evolve independently. Modifications in the interface agent would not imply a modification of the base application.

In this paper we present an object-oriented framework that will enable an interface agent to observe the actions the user performs in a software application without modifying in any way the application's source code. This framework can be applied even when the developers of the interface agents do not have the application's source code available as the linking between the agent and the application is done in running time and not in compilation time. We do not face here the problem of assisting users.

In our framework, in order to become aware of the user's interaction with the application, an interface agent intercepts the user's clicks, keys pressed, mouse movements, between others GUI (Graphical User Interface) events. The kind of events the agent would be interested in depends on the kind of application. For example, in a calendar application, keys pressed and mouse clicks would be interesting events, and not mouse movements or

drags, which in turn would be interesting in a drawing application.

Regardless of programming language or implementation, most modern direct manipulation interfaces are structured around an interpreter called an "event loop" that accepts mouse and keyboard input, tries to determine what functionality the input is requesting, changes the application data structures, and updates the screen display.

As a specific implementation of detection of the user's actions will be dependent on the programming language in which the application is implemented, we worked as an example with applications written in Java².

The basic idea of our framework is to wrap the Java application and observe all the event activity in the Java Abstract Window Toolkit (AWT) event queue. The agent then filters out the events that are not considered actions within the application. Then, following the idea of an application as a set of tasks (tools to accomplish some goal), and making use of the detected GUI events, the agent has to determine which task of the application is associated to each of them. To do so, and having in mind that the agent code is disengaged from the application code, the agent has to have some model of the application. This model should include the tasks the application provides, and the steps necessary to accomplish each of them. These task models include the tasks the application provides, and the steps necessary to accomplish each of them, including temporal restrictions in the order of execution. With this information available the agent will be able to know which task the user is performing and which subtasks he has to perform to complete the top level task. With this information on hand the agent will be able to help the user in the completion of the tasks offered by the application. Figure 2 shows an overview of what we explained

² <http://java.sun.com/> [11 July 2007]

above.

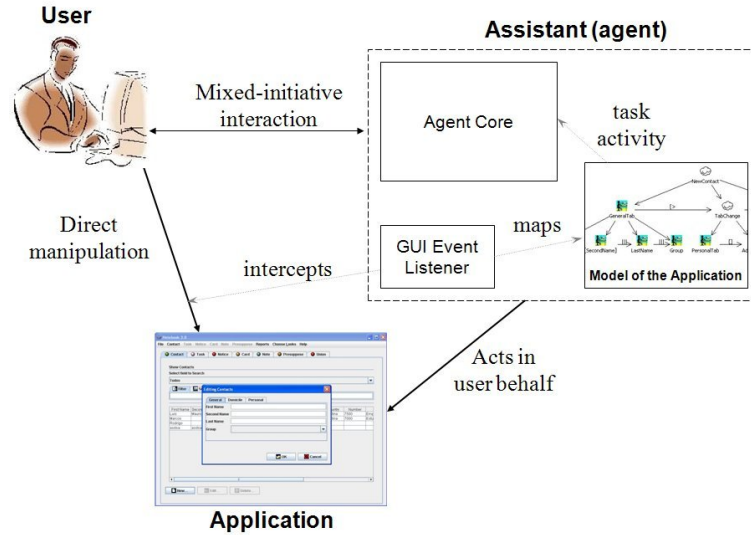


Figure 2: Overview of our approach

This paper is structured as follows. In Section 2 we present some previous efforts facing the problem of integrating interface agents with conventional applications. Then, in Section 3 and Section 4 we present our approach to the problem. In Section 5 we give some implementation details of our framework and in Section 7 we present a case study. Finally in Section 7 we present our conclusions.

2 Monitoring the user's actions

The problem we tackle in this paper is how to become aware of the tasks the user performs in a software application, without modifying the application itself. We are only aware of two efforts towards this objective, which are presented next.

2.1 The spying technique

Pachet and Giroux (2) proposed a technique for grafting an “observation” system onto arbitrary object-oriented applications, without modifying their code. This technique is based on the notion of *spy* of a particular object that may be inserted in object-oriented systems in a non-intrusive manner, and may track incoming messages to arbitrary objects. A spy is an object that may be inserted in a program in order to detect all incoming messages to a given object, without modifying the program's semantics. They presented an implementation of their ideas in the Smalltalk programming language. In practice, this is realized by a combination of capsule programming (3) together with mechanisms for swapping objects identities. The main idea behind capsules is their ability to redefine message reception at the instance level.

The capsule mechanism introduced by Pascoe consists in substituting capsule objects to spied objects. Practically, capsules require a modification of methods that actually create the objects to be spied. Encapsulation therefore may not be performed on existing code without modification. Pachet and Giroux proposed to automate the creation of capsules, by a mechanism which automatically encapsulates objects, based on the systematic use of the Smalltalk primitive *become*:. This primitive message swaps the internal address of two arbitrary objects. Here, the idea is to encapsulate objects by making them physically "become" spies, which in turn point to the original object. With this method, they can encapsulate objects without redefining existing code.

One drawback of Pachet and Giroux approach is that spies cannot intercept messages sent internally by an object to itself (what they call "the *self* problem"). A related problem is the "reverse self" problem: certain messages may be intercepted by spies when in fact they should not, because they are mere 'echoes' of already intercepted messages. To solve this problem, spies should be designed so that they intercept only one message at a time, using

stack introspection. On the other hand, *classes* cannot be spied, for technical reasons, which prevent an easy detection of the creation of new objects. Finally, the processing of spied information by spies may be a burden for the computer which is running the application been spied.

2.2 Marionette Strings

Henry Lieberman (4) presented an approach called “marionette strings”. In his approach the agent is given a set of “strings” corresponding one-to-one with user actions in the interface and can “tug” on the strings to make the program perform.

He based his studies in scripting languages like AppleScript and Visual Basic. Scripting languages take advantage of the following observation. Although most commercial applications are not written so that their functionality is accessible by another program, they must provide some way for the user to access that functionality through input, typically by selecting some item from a menu, clicking on some icon, or typing some command. Thus, if another program can “fool” the application into thinking that the user typed or clicked something, it too could access the functionality of the application.

The marionette strings are implemented by stuffing an object representing user input into the application's input buffer. The application then processes it “as if” the input was produced by user interaction.

It is necessary for the agent to have an object-oriented model of the data in the application it is dealing with. Since the agent is supposed to act as a proxy for the user, one could think of this model as the agent's “mental model” of the application. As in a user's mental model of the application, the model need not consist of all the objects in the application, but at least those that are directly visible and of concern to the user for specific

purposes in the interface, and the minimal set of internal objects that affect the application.

This approach is not completely transparent for the application in the sense that it requires a minimum of advance planning on the part of the application developer: the application writer must agree on a set of operations to "export" as events, which determines what is accessible to the agent, and the agent must agree to "import" the events.

There is the problem of deciding at how fine a level to do this. In the limiting case, every time the program does anything, it must create an event corresponding to that action, and be prepared to execute that action in response to an event requesting it from the agent. In that case, the application effectively becomes a program written in the event language rather than the underlying programming language. The "application code" itself becomes nothing but an interpreter for the code written in the event language. At worst, the interpretation overhead would slow the application, and at best this would require two versions of everything, an interpreted and a compiled version.

To take a concrete example, should we include mouse movements in the protocol that an agent should be able to receive? If this is the case, then we are obligating every application to report every time the mouse moves, which is potentially inefficient, and wasteful in the case the agent does not need this information. If not, we are preventing all agents from ever tracking the mouse, and it seems like at least some agent might in fact want this ability.

3 Observing the user's actions

The detection of the user's actions will be dependent on the programming language in which the application is implemented. In this work, we deal with applications written in Java.

We follow an approach similar to that presented by Lieberman, but we try to overcome

the limitation of the application being prepared to report the set of events that the agent may be interested in.

Unlike Pachet and Giroux approach, we do not intercept incoming messages to a given object, as in Java we do not have a direct way of encapsulating objects by making them physically "become" other object. Instead, we intercept the events generated by the widgets in the GUI of the application by observing the Java Event Queue.

In Java, every time the user types a character or pushes a button in the user interface of a software application, an event occurs. Any object can be notified of the event. All the object has to do is to implement the appropriate interface and be registered as an event listener on the appropriate event source.

For example, to detect when the user clicks a button on the screen, as shown in Figure 3, the application must have an object that implements the *ActionListener* interface. The program must register this object as an action listener on the button (the event source), using the *addActionListener()* method. When the user clicks the onscreen button, the button fires an action event. This results in the invocation of the action listener's *actionPerformed()* method (the only method in the *ActionListener* interface). The single argument to the method is an *ActionEvent* object that gives information about the event and its source.



Figure 3: Example of the event generated when pressing a Button in Java

Swing components can generate many kinds of events. Table 1 lists a few examples of Java events and their corresponding listener type.

Act that results in the event	Listener Type
User clicks a button, presses Enter while typing in a text field, or chooses a menu item	ActionListener
User closes a frame (main window)	WindowListener
User presses a mouse button while the cursor is over a component	MouseListener
User moves the mouse over a component	MouseMotionListener
Component becomes visible	ComponentListener
Component gets the keyboard focus	FocusListener
Table or list selection changes	ListSelectionListener
Any property in a component changes such as the text on a label	PropertyChangeListener

Table 1: Example of some Swing events, and their associated Event Listeners

The event model in Java is quite powerful and flexible. Any number of event listener objects can listen for all kinds of events from any number of event source objects. For example, a program might create one listener per event source. Or a program might have a single listener for all events from all sources. A program can even have more than one listener for a single kind of event from a single event source. We will make use of this capability to add the interface agent as a listener to all the event activity in the application that the agent may be interested in, without having any knowledge of the listeners the application has to provide its own functionality. Figure 4 schematize what we have just discussed. Each event is represented by an object that gives information about the event and

identifies the event source. Event sources are often components or models of those components.

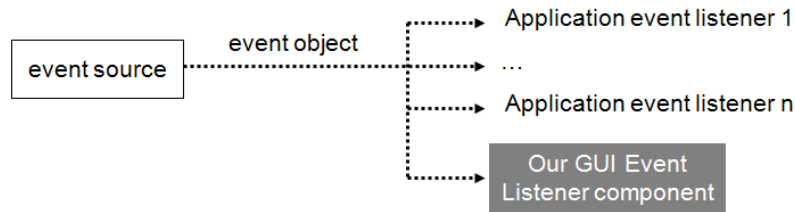


Figure 4: Our approach to "observe" the user's actions in the application

4 Mapping actions to tasks

Once the agent has detected an action of the user on the application GUI, it should map it to the task that it represents in the context of the application. Having in mind that we are facing the problem of developing an interface agent for an existent software application for which we do not have the source code available, the agent have to have a model of the tasks that can be performed in the application.

We make use of the application's *task model* to detect how each user action contributes to the current task. A task model is a representation used to specify the possible tasks the user can perform in a given application. The task model is an important source of knowledge for the agent, because it contains information about all possible tasks that the user can perform.

A task defines how the user can reach a goal in a specific user interface of an application. The goal is either a desired modification of the state of the system or a query to it. Moreover, we use the task model further to associate each task the action of the user over the application GUI that active it. Users perform tasks, which are structured set of activities

(5) in order to achieve higher level goals. Tasks can be further decomposed to represent lower level sub-goals. This notion of decomposition naturally results in tree-like structures and thus in a hierarchical representation of the model. More recently, task model representations such as *Concur Task Trees* (6) have enabled the distinction of abstract, cognitive, interactive and system activities as well as the possibility to model temporal aspects of activities when specifying a task model. The typical characteristics of a task model include its hierarchical structure, the task decomposition, and sometimes the temporal relationship between elements. The best known techniques for task modeling are based on hierarchical task analysis. These general models contain information about the structuring of the tasks into subtasks and about the order of their execution.

There are many different task model representations, which differ in their type of syntax (textual or graphical), the level of formality and the richness of operators offered to designers. Hierarchical Task Analysis (HTA) (7) is a task analysis technique in which models are represented textually or graphically with structured numbering corresponding to the various goals and decomposed actions. The Goals, Operators, Methods and Selection rules (GOMS) (8) model was developed as a model for predicting human performance while interacting with a system; the first models of this family only considered sequential tasks. Groupware Task Analysis (GTA) (9) was developed for the modeling of complex tasks in cooperative environments; it mainly focuses on the analysis phase aiming at eliciting requirements that can be important in the user interface design. Concurrent Task Trees (6) allows the specification of tasks, roles and objects as well as a task hierarchy and the temporal relationships between defined tasks. Operators are used to link sibling tasks, on the same level of decomposition. *Méthode Analytique de Description de tâches* (MAD) (10) combines structured interviewing and modeling techniques focusing explicitly on the

hierarchical relation of tasks. The main concepts of MAD include the task, action and structure. Task Knowledge Structure (TKS) (11) is based on the assumption that people possess knowledge structures in their memory that relate to tasks. The technique represents this conceptual knowledge to describe the roles, goals, plans and procedures comprised of actions and objects. Finally, User Action Notation (UAN) (12) is a textual notation designed to formalize the communication between the User Interface designers and the development team when specifying and analyzing details of technology.

It is not the purpose of this thesis to make a complete analysis of different approaches to task modeling. We make use of a task model as a mean to associate user actions to application tasks and to know the tasks the user is expected to perform according to temporal restrictions within the current top level task, thus we choose to use the Concur Task Trees notation as it is a well-known and widely used task model representation by the HCI community. In the following section we expose further arguments for using Concur Task Trees as a task model representation.

4.1 ConcurTaskTrees

In this section we list the main features of Concur Task Trees notation that made us select it to specify the task models in our approach:

- An intuitive hierarchical structure which provides a large range of granularity allowing large and small task structures to be reused, and which enables reusable task structures to be defined at both a low and a high semantic level.
- A graphical syntax that is easy to interpret and which reflects the logical structure in a tree-like form;

- Concurrent notation: operators for temporal ordering are used to link subtasks at the same abstraction level. It is important for designers to express clearly the logical temporal relationships because such ordering should be taken into account in the user interface implementation to allow the user to perform at any time the tasks that should be active from a semantic point of view.
- It is focused on activities, thus it allows designers to concentrate on the most relevant aspects when designing interactive applications that encompass both user and system-related aspects avoiding low levels implementation details that at the design stage would only obscure the decisions to take.
- It defines different categories of tasks that indicate how the performance of the task is allocated.
- It allows each task to be associated with the objects that have to be manipulated to support their performance. These features allow designers to describe a greater range of behaviors than those that can be described by HTA or GOMS. UAN introduced richer sets of operators, but it supports only asynchronous tasks. Furthermore, UAN uses a textual notation with no tool support which makes it unsuitable for developing and analyzing large specifications.

On the other hand, Concur Task Trees has an associated environment (CTTE) (13) aimed at being used to support the design of interactive applications, for teaching purposes, and to produce interactive documentation of applications. CTTE has been used in many projects, as listed in the authors' web page³.

³The complete list of external uses of CTTE can be found in

4.2 Task modeling

Tasks models in our approach are constructed manually. ConcurTaskTrees environment (CTTE) is very useful tool to construct task models and to test if they have the behavior that is expected. More information about the use of CTTE can be found in its home page⁴.

Each task of the model has a tree-like structure, as can be seen in Figure 5, where leaves are simple tasks and internal nodes represent tasks composed by other (simple or composed) tasks. The information associated to tasks include the name used to identify it, its category and type, frequency of use, some internal description, whether it is iterative and/or optional, associated objects and time performance. Iterative tasks are indicated with an asterisk (T*) by the name of the task. Optional tasks, on the other hand, are put in brackets ([T]).

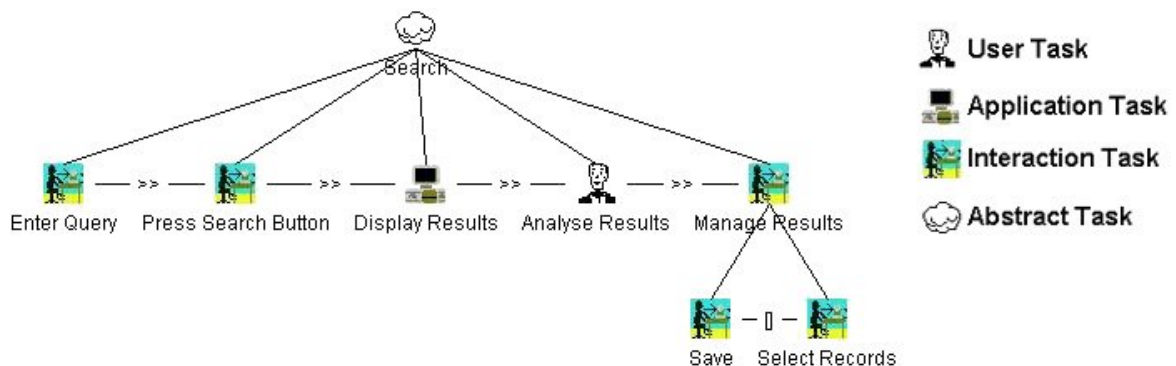


Figure 5: Example of a Concur Task Tree task model for a general “Search” task.

Concur Task Trees distinguish four categories of tasks depending on the allocation of

<http://giove.cnuce.cnr.it:8443/CTTE/Externaluse.jsp> [11 July 2007]

⁴ <http://giove.isti.cnr.it/ctte.html>

their performance; each category is represented with a different icon (Figure 5). *User Tasks* are performed entirely by the user. They require only cognitive activities without interaction with the system, such as selecting a strategy to solve a problem or analyzing the content of some information received and evaluating whether or not it is useful. In Figure 5 "Analyze Results" is a User Task. Thus, user tasks are associated with some processing performed by the user on information received from the environment. *Application Tasks* are completely executed by the system. They receive information from the system and they can supply information to the user. They are activated by the application. For example, compiling a program and sending messages when some errors are detected, or displaying results of a query, such as "Display Results" in Figure 5. *Interaction Tasks* are performed by the user interactions with the system. These interactions are activated by the user. Examples are editing a diagram or formulating a query to a data base. Most of the tasks in Figure 5 are interaction tasks. Finally, *Abstract Tasks* are tasks that have subtasks belonging to different categories, and their performance does not completely fall into one of the three previous cases. The root of the tree in Figure 5 falls in this category.

In our approach, we cannot access to the user's mental state, so we will not be able to detect user tasks. Some application tasks can be detected, providing that they modify the state of some component in the GUI of the application. Interaction tasks are clearly of interest to our purpose as the user generates system events while interacting with the application that can be observed by the agent. So this kind of tasks will have a corresponding GUI event that indicates its performance. The mapping of events to interaction tasks is done manually when constructing the task model of the application.

While the category of the task indicates who is involved in its performance, the type of each task allows the grouping of them according to its semantics. Each task category has its

own set of types. In the interaction category, examples of task types are selection (the task allows the user to select a piece of information), control (the task allows the user to trigger a control event that can activate a functionality), and editing (the task allows the user to enter a value).

The objects that can be associated to a task are entities which are manipulated to perform it. There can be either user interface (or perceivable) or domain application (or internal) objects. The former kind of objects is items which users can interact with using their senses, for example menus, icons, windows, etc. On the other hand, internal objects are entities which belong to the application and which need to be mapped onto perceivable objects to be presented to the user. Examples of internal objects are the state of a request for a data base, the data base itself, etc. We will make use of objects to map user actions to application tasks.

4.3 Temporal constraints

CTT offers also a complete set of operators that allow us to define restrictions in the order the subtasks are performed. In Figure 5, for example, subtasks of "Search" have to be performed sequentially, and once the user analyzed the results, he can either save them or select a record to inspect (only one of that tasks can be performed).

The operators that we can use to describe temporal relationships among tasks in the same level of decomposition are:

- Choice ($T1 \square T2$): It is possible to choose from a set of tasks and, once the choice has been made, the task chosen can be performed and other tasks are not available at least until it has been terminated. This operator is used when the

user has to choose from various tasks.

- Order independence ($T1 \models T2$): both tasks have to be performed; any of them can be the first task, but once one of them is started, then it has to be finished before starting the other.
- Concurrency ($T1 \parallel T2$): as with the order independence operator, both task have to be performed, but in this case there are no constraints in the order of execution of its subtasks;
- Concurrent with info exchange ($T1 \parallel [] T2$): the tasks have to synchronize in order to exchange information. For example, in a word processor application where editing a file and scrolling its content can be performed in any order, these tasks exchange information when they are performed because it is possible to edit only the information that the scrolling has made visible.
- Deactivation ($T1 [> T2]$): when one action from the second task occurs, the first task is definitively deactivated. This concept is often used in many user interface implementations when the user can deactivate the option of performing a set of tasks and enable a new set of possible tasks by performing a specific action (for example by selecting a button). A more common example is when the user accepts or cancels a dialog.
- Suspend/Resume ($T1 | > T2$): This operator gives T2 the possibility of interrupting T1 and then when T2 is terminated, T1 can be reactivated from the state reached before the interruption. For example, the editing text task which, in some applications can be suspended by a modal printing task, and once the printing task is accomplished then editing can be carried on from the state

reached beforehand. This operator can be used also to model a type of interruption.

- Enabling ($T1 \gg T2$): only when the first task is terminated, the second task is activated. For example, we have to select a file before editing it.
- Enabling with information passing ($T1 [] \gg T2$): in this case we want to highlight that when T1 task terminates it provides some value for task T2 besides activating it;

4.4 Example

Figure 7 shows a segment of a task model for the calendar task of editing a contact which GUI is shown in Figure 6. The relations between nodes in the same level denote different kinds of temporal restrictions in the execution order of the tasks.

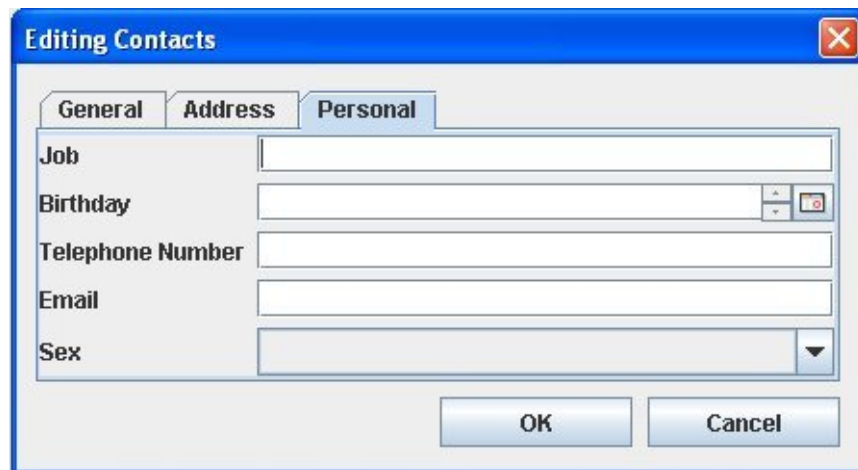


Figure 6: Screenshot of "Editing contact" task in a calendar application

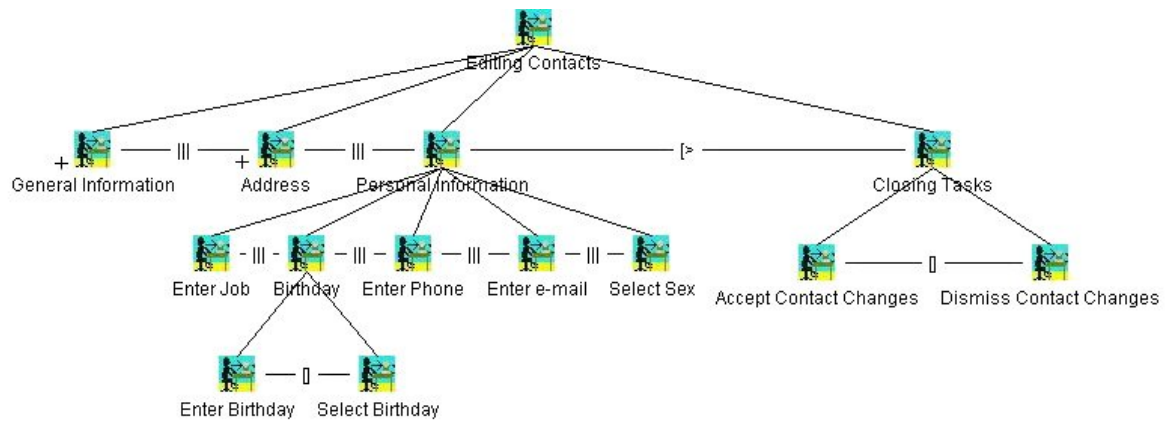


Figure 7: Task Model related to screenshot in Figure 6

We have a task named "Editing Contact", whose subtasks (or steps of the task) are "General Information", "Address", "Personal Information" and "Closing Tasks". The first two tasks have further subtasks, but they have been folded (as indicates the "+" sign by the task icon) aiming for clearness.

"General information", "Address" and "Personal Information" tasks can be executed concurrently, in any order (||| is the concurrency operator). However, any of those tasks are terminated ([> is the disabling operator) by the execution of "Closing tasks", either "Accept Contact Changes" or "Dismiss Contact Changes" ([] is the choice operator). "Personal Information" has five subtasks that can be performed concurrently, where "Birthday" can be performed either by entering the date manually ("Enter Birthday") or selecting it from a calendar ("Select Birthday").

"General information", "Address" and "Personal Information" tasks will have as interface objects the panels widgets associated to each tab in the GUI. "Enter Job", "Enter Birthday", "Enter Phone" and "Enter e-mail" will be associated to the corresponding TextFields, "Select Birthday" will be associated to the calendar button, "Select Sex" to the ComboBox, and "Accept Contact Changes" and "Dismiss Contact Changes" to the "Ok"

and "Cancel" buttons, respectively.

5 Implementation details

In the previous section we described how we deal with the detection of the user's action and its mapping to a task of the application. In this section, we will briefly describe how we designed and materialize these aspects in an object oriented framework, its implementation in the Java programming language, and how these components should be instantiated to be adapted to a particular software application.

Figure 8 shows an outline of our framework. The observation of the user's actions over the application is done by a single Java class named `GUIEventListener`. `GUIEventListener` registers the event listeners using a class named `SwingEventMonitor` provided by the Java Accessibility Utilities⁵ package. This package aims at the construction of technologies that assist users with some kind of impairment, such as screen readers, speech recognition systems and Braille terminals. However, it is very useful to our purposes of accessing to the information of the GUI components that are being presented to the user by the application, and to the interaction of the user with those components.

The Java Accessibility Utilities help assistive technologies take advantage of applications developed using the Java Accessibility API⁶. The Java Accessibility API is one of the core foundation services in the Java Foundation Classes⁷, a comprehensive set of graphical user interface components and foundation services designed to simplify deployment of Internet, intranet and desktop applications. An application developer can use

⁵<http://java.sun.com/products/jfc/jaccess-1.3/doc/index.html> [11 July 2007]

⁶<http://java.sun.com/products/jfc/swingdoc-api/javax/accessibility/package-summary.html> [11 July 2007]

⁷<http://java.sun.com/products/jfc> [11 July 2007]

the Java Accessibility API directly or indirectly via a toolkit such as Swing that implements the Java Accessibility API. By doing so, developers will be able to create Java applications capable of interacting with assistive technologies such as screen readers, speech recognition systems and Braille terminals. Accessibility enabled Java applications are not dependent on machines that require assistive technology support; rather these applications will run on any Java platform with or without assistive technologies.

In order to provide access to Java applications, however, an assistive technology requires more than the Java Accessibility API: it also requires support in locating the components that implement the API as well as support for being loaded into the Java Virtual Machine, tracking events, and so on. The Java Accessibility Utilities provide this assistance.

This utility package provides the necessary support for assistive technologies to locate and query user interface objects inside a Java application running in a Java Virtual Machine. It also provides support for installing 'event listeners' into these objects. These event listeners allow objects to learn about specific events occurring in other objects.

This support provides a list of the top-level windows of all of the Java applications, an event listener architecture to be informed when top level windows appear (and disappear), and means for locating the window that has the input focus, locating the mouse position, and inserting events into the system event queue. The Java Accessibility Utilities provide an alternate system EventQueue in the class EventQueueMonitor that implements the functionality described previously.

The Java Accessibility Utilities include three classes for monitoring events in the Java Virtual Machine. The first class, AWTEventMonitor, provides a way to monitor all AWT events in all AWT components running in the Java Virtual Machine. This class essentially

provides system-wide monitoring of AWT events, registering an individual listener for each AWT event type on each AWT component that supports that type of listener. Thus, an assistive technology can register a `char`'Focused listener'`char`' with `AWTEventMonitor`, which will in turn register a `char`'Focused listener'`char`' with each and every AWT component in each and every Java application in the Java Virtual Machine. Those individual listeners will funnel the events they hear about to the assistive technology that registered the listener with `AWTEventMonitor` in the first place. Thus, whenever a component gains or losses focus (e.g. the user hits the TAB key), the assistive technology will be notified.

The second class, `SwingEventMonitor`, extends `AWTEventMonitor` to provide additional support for monitoring the Swing events supported by the Swing components. Since `SwingEventMonitor` extends `AWTEventMonitor`, there is no need to use both classes, and we only register our listeners with `SwingEventMonitor`.

The third class, `AccessibilityEventMonitor`, provides support for property change events on Accessible objects. When an assistive technology requests notification of Accessible property change events using `AccessibilityEventMonitor`, the `AccessibilityEventMonitor` will automatically register Accessible property change listeners on all the components. In addition, it will detect when components are added and removed from the component hierarchy and add and remove the property change listeners accordingly. When an Accessible property change occurs in any of the components, the `AccessibilityEventMonitor` will notify the assistive technology.

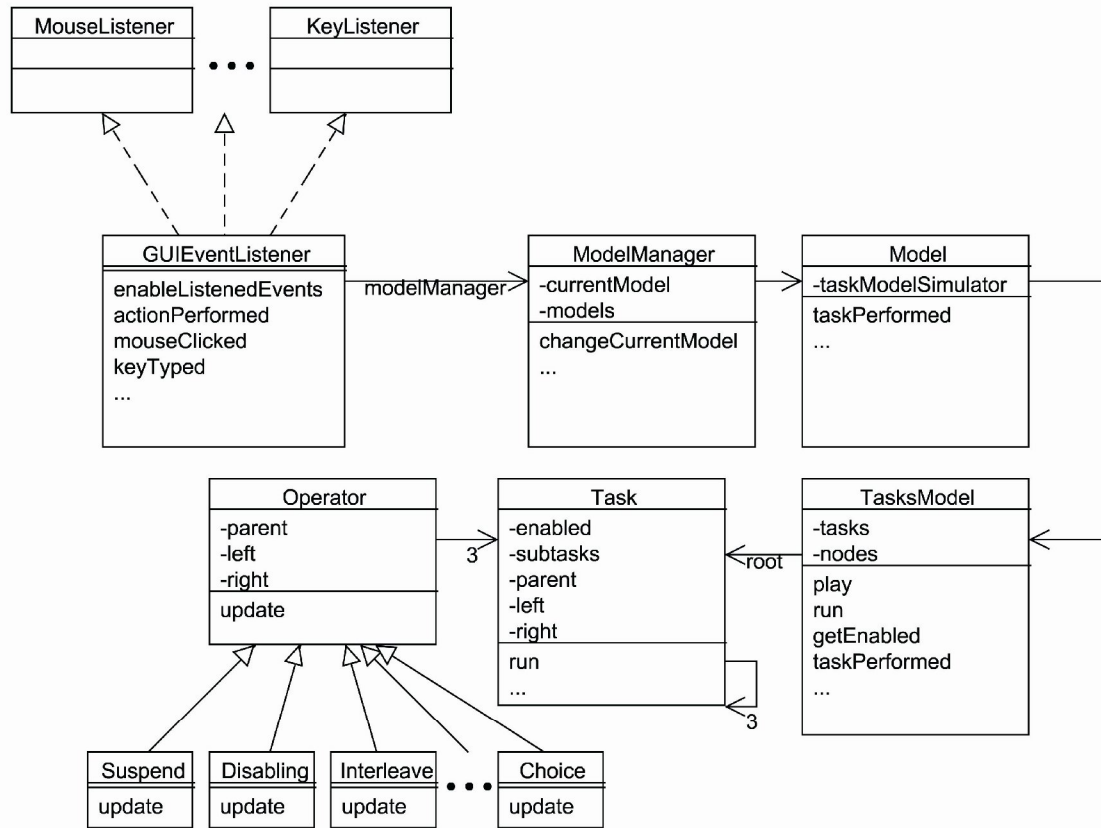


Figure 8: Class Diagram outline of our framework for detecting the user's tasks

In our framework, the implementation of `GUIEventListener` is quite simple. It implements all the Java interfaces in the packages `javax.awt.event.*` and `javax.swing.event.*` (such as *MouseListener* and *KeyListener*) with a default empty implementation to make any kind of event listener available to the base application. `GUIEventListener` class should be extended redefining the methods that the agent is interested in listening to, according to the base application.

For example, if the agent needs to monitor the keys the user presses when it uses the application, a subclass of `GUIEventListener` should redefine the methods provided by the interface `KeyListener`: *keyTyped(KeyEvent theEvent)*, *keyPressed(KeyEvent theEvent)* and *keyReleased(KeyEvent theEvent)*.

GUIEventListener class has a method for each possible interface event that can occur in a Java application. Moreover, the method *enableListenedEvents()* is used to read from a configuration file which events will be listened by the agent, not to burden the system listening to events that are not interesting for a particular agent. GUIEventListener class has a reference to a model manager to which it will inform about the GUI event that occurred in the application.

Concrete subclasses of GUIEventListener class should inform to the instance of ModelManager kept in this class every time it listens a GUI event. ModelManager (see Figure 8) class keeps a hash table with all the tasks models of the application (instances of the Model class), indexed by its name and a reference to the current model according to the actions performed by the user so far. The main method in this class is *changeCurrentModel(String name)* which updates the current task model according to the task the user is performing.

This class should also be extended to implement a model manager of a particular application. This subclass should reflect the peculiarities in the way of acting of the application, and the transition between its tasks models, invoking the method *changeCurrentModel*.

The Model class represents a task model of the application. It keeps a hash table of properties or attributes of the model, where each entry corresponds to a <attribute name; attribute value> pair. These attributes will register the values entered by the user in the GUI widgets so that they are available to the interface agent if needed. The main method in the Model class is *taskPerformed(String taskName)*. This method is in charge of notifying that a task in the application has been performed to any interested object in the agent. To do so, the Model class extends the Observable class, following the Observer-Observable design

pattern. This way, we can register that any number of Observer objects is interested in the execution of a task, without having any knowledge about them in the Observable class (Model in this case). Other components of the interface agent implementation will be waiting for this kind of notification to get aware of the tasks the user is performing.

A very important attribute of this class is the *taskModelSimulator*. The task model simulator is used to keep track of the subtasks the user should perform to complete a task in a task model. To have this information in any given moment, we follow the idea of the *Task Model Simulator* presented in CTTE. CTTE, the tool supporting CTT, gives the possibility of simulating the task model. This means that it is possible to interactively select one task and ask the system to show what tasks are enabled after the performance of that task. When the CTTE simulator is started, it displays the list of tasks that the user can perform according to the temporal constraints present in the model. Then the user can simulate the execution of a given task, and the simulator updates the set of enabled tasks, to be consistent with the task "performed" by the user. Even if this functionality is directly useful to our purposes, CTTE is distributed within a graphical tool and its functionality cannot be accessed via a Java API. For this reason, we implemented our own Task Model Simulator; its related classes are shown in Figure 8.

The task model simulation starts when the user performs a task of any given task model by invoking the *play()* method in the class *TasksModel*. This method initializes the data structures that represents the task model, and send an activation message to the root task. This activation message is propagated down in the tree and, according to the order restrictions in the model, the set of enabled tasks is generated. Each task is represented by an instance of the *Task* class, and each operator by a subclass of the *Operator* class. We have a class for each different operator in CTT notation, because the way they respond to

the execution of a task is different according to the semantic of the operator.

We used here the Observer-Observable design pattern again. Each operator observes three tasks objects: the task at its right, the task at its left and the parent task of those two tasks. When the state of activation or execution of some of those observed tasks changes the operator is in charge of modifying the state of the other tasks keeping this way the semantics of the model.

The list of tasks that are enabled in a given moment is returned by the method named *getEnabled()*. Then the execution of a task can be indicated by invoking the *run(String taskName)* method, where *taskName* is the name of the task to perform. This method will modify the state of the task and this change will be noticed by the corresponding operator, which will take the appropriated actions for the model to reflect such change. Every time the *run* method is invoked, the *taskPerformed* method is invoked as well, to notify external listeners about the execution of the current task.

The task models used by our framework can be generated using CTTE or by an equivalent application developed by us to test the implemented simulator. Both of these applications allow exporting the model as an XML description. Our framework is able to import and interpret this description and to generate the corresponding tasks models for being used by the *ModelManager* component.

6 Case Study

In the previous Sections we presented our approach to enable an interface agent to recognize the tasks a user performs in a software application. We proposed the use of a GUI event listener approach to “observe” the actions of the user in the application without having to modify the application itself and a task model of the application to know what

task the user is performing in any given moment and which subtasks it has to perform to complete a top level task.

To illustrate and evaluate our proposed approach we used a Java application called Agenda 4.0, that provides various functions for managing personal information such as contacts, tasks, events, multiple purpose cards, notes, budget and email. In this Section we will explain how we can instantiate the framework presented in Section 5 to work with this specific application.

6.1 Agenda application

The Agenda application we used as the base system for detecting the user's intention enables a user to organize and visualize personal information. By using such a system a user can manage an address book with contacts, that is add a new contact, edit the information of a contact, organize contacts into groups, send an email to a contact; he can schedule events, delete events, edit the information of an event, add priorities to events; he can keep a list of to-do tasks and a list of notes; he can organize a set of cards with specific information such as web pages, cooking recipes, etc.; he can register incomes and outcomes in his budget; and he can generate reports of any of the functionality provided by the application. Figure 9 shows an screenshot of this application.

Each functionality offered by Agenda application is distributed in a different tab on the top of the window frame. As we can see in Figure 9, we have a tab for contacts, tasks, events, cards, notes, budget, union and email. The union tab is simply a conjunction of all the other functionalities and it is used to view in only one screen all the information registered by the user. On the other hand the email tab offers the user the basic functionality for sending and receiving email. Within each tab, except for email tab, Agenda 4.0 presents

a list with all the records created by the user for the functionality in question, for example, in Figure 9, the user is in the tab named “Contact” and the application presents a list with all the contacts currently registered. The user has the possibility of filtering or searching a specific item in this list by any of its fields. This task is done by the “Filter” and “Search” tab, on top of the list of items, and by a combo box that allows the user to select the field to filter or search by. For example, if the user is working with contacts, he could filter the contacts which has a value of “Tandil” in the “City” field. The difference between filtering and searching is that by filtering, items that do not satisfy the filter are not shown in the list, and by searching the application simply select the first item satisfying the search keywords. Finally, the user can create a new item, edit an existent item and delete one or more items simultaneously, by using the buttons in the bottom on the screen named “New...”, “Edit...” and “Delete...”. Figure 10 shows a screenshot of the window presented to the user when he presses the “New...” button in the “Contact” tab.

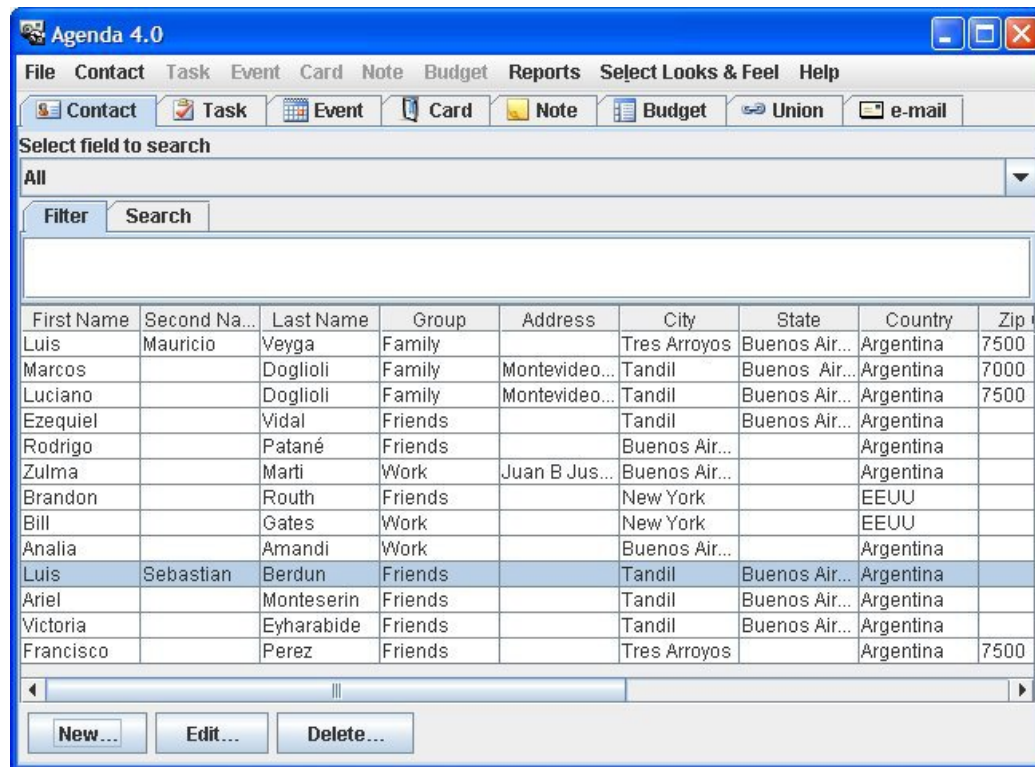


Figure 9: Screenshot of Agenda 4.0 application

The “File” menu allows the user to personalize the lists of options presented to the user when creating a new item. For example, when adding a contact, the user can select a group for the contact from a list of possible groups (see Figure 10). The items in this list can be added, edited or deleted from the “Properties” sub-menu in the “File” menu, as shown in Figure 11.

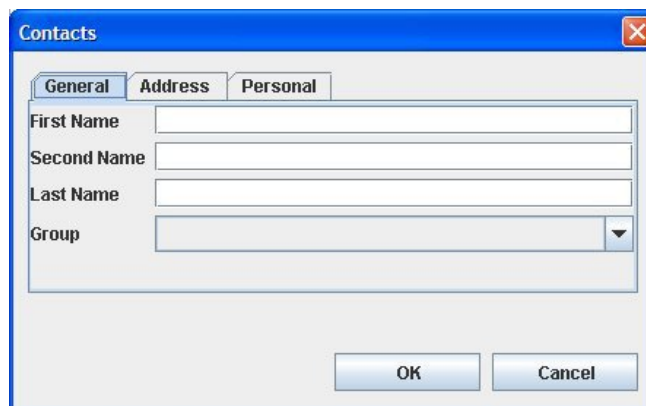


Figure 10: New contact addition in Agenda 4.0

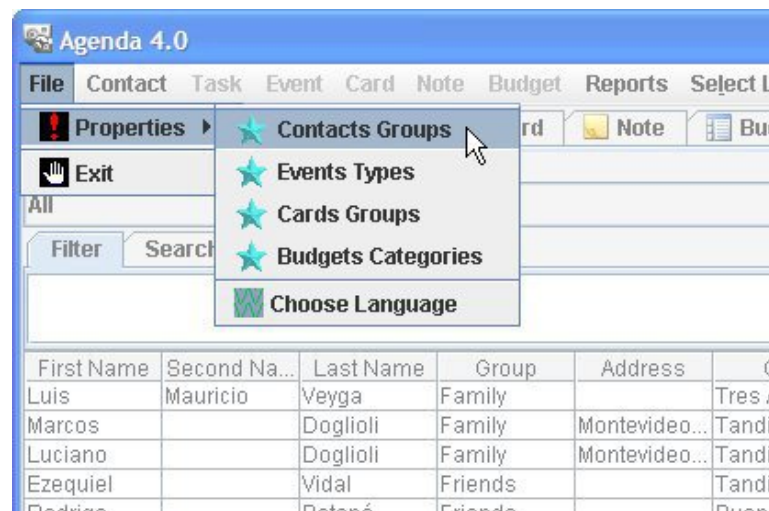


Figure 11: Agenda 4.0 Properties menu

The calendar domain is a well known and easy to communicate domain in which interface agents can be applied. An interface agent in this domain can assist users with tasks such as suggesting meeting places, dates and times for a new event the user is scheduling, suggesting the participants of a meeting, notifying the user when an event is about to happen and warning about overlapping events and about an unusual event scheduled by the user. To provide this functionality, the agent should be aware of the task the user is performing because it gives the context in which the user is moving through. This context will allow the agent to infer the intention of the user, so that it can provide assistance to him if needed, or it can collaborate with the user. A complete development of such an agent is out of the scope of our work. In the remaining of this Section we will center on how the agent becomes aware of the tasks the user performs in the base application.

In the remaining of this section we will detail the steps that should be followed when we want to apply our model to a particular application, using Agenda 4.0 as an example.

6.2 Analyzing events of interest

The first step we should take to model the application following our proposed model for observation of a user's activities is to analyze the widgets in the application's user interface.

Agenda 4.0 functionality is organized in tabs that correspond to a `JTabbedPane` component of Swing framework. Within each tab, the application presents the items in a `JTable` component. To add, edit or delete an item, we have to interact with `JButton` components. Filter and search functionality is also selected with a `JTabbedPane`, the text by which the application will filter or search is typed in a `TextField`, and the field for which the user will filter or search items is selected from a `JComboBox`.

When adding or editing an item, the application presents a `JDialog` with various `TextBox`, `JComboBox` and `JCheckBox` components. To select a date, a composed component is used which is not in part of the Swing framework offered by Java. However, this widget is compound with other simpler widgets which are included in Swing.

Once we have recognized the Swing widgets in the GUI of the application we have to identify the events that are dispatched when the user interacts with them so that we can override the corresponding event listeners' methods in `GUIEventListener` class provided by the framework implementing our model. Recall that `GUIEventListener` class implements with a default empty method all the interfaces provided by Swing for listening to the events in the application's interface. These methods receive as an argument an object of type `Event` from which it is possible to obtain the source of the event, that is the widget from which it comes from. The source of the event is used with many purposes, such as to obtain the name of the component and the information entered by the user.

For Agenda 4.0, we extended `GUIEventListener` with a class named

AgendaGUIEventListener. In this class we override the following event listener methods to detect the events generated by the application's GUI:

- *actionPerformed*: is invoked by the application when the user presses any of the buttons in the agenda, for example the buttons labeled “New...”, “Edit...”, “Delete...”, “OK”, and “Cancel” in any functionality offered by the agenda. This method is also invoked when the user interacts with a combo box to select an item in any dialog of the application.
- *mousePressed*: is used to detect when the user clicks on a different tab, when the user selects an item in a menu and when the user selects an item in a table (which lists the items of any functionality of the agenda)
- *focusLost*: is used to detect when the user had finished editing a text box or a text area. We do not use key listener here, because a key event is thrown with every key pressed by the user, and we are only interested in the final text, not in individual characters.
- *keyTyped*: is used to detect the keys pressed by the user when he is searching or filtering information. In this case, the application reacts with each key typed by the user. For this reason we use keyListener here nor focusLost as in the other text fields in the application.

6.3 Modeling the application

The next step is to build the set of task models of the application. Figure 12 shows the task model that corresponds to the screenshot in Figure 9 for managing contacts. In our sample application, the task models corresponding to the remaining functionality are very

similar to the task model presented for managing contacts.

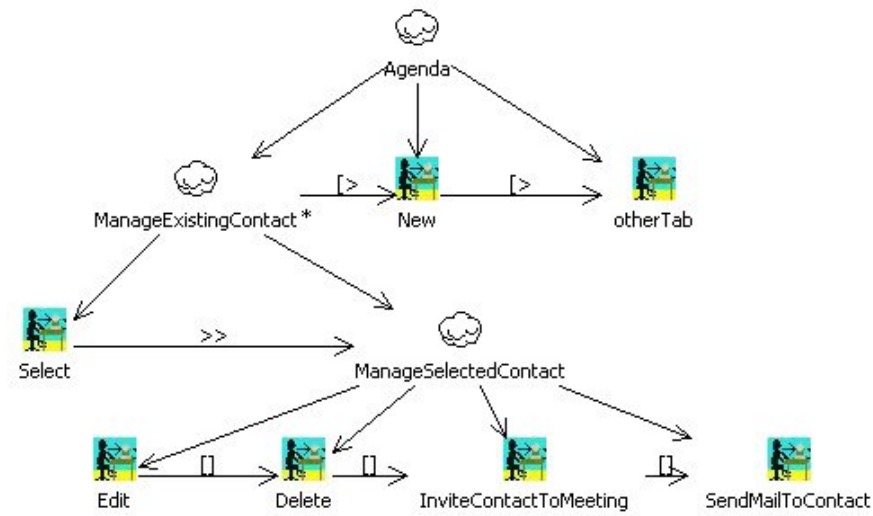


Figure 12: Task model for contacts tab in Agenda 4.0

The root task represents the Agenda application itself. The application presents to the user the list of existing contacts. So the user can manage existing contacts, add a new contact (“New” task) or change to another functionality offered by the agenda by selecting another tab (summarized by the “otherTab” task). “ManageExistingContacts” is an iterative task (as indicated by the * beside its name) which groups the tasks that can be performed within this screen of the application. “ManageSelectedContact” task groups the tasks that need a contact to be selected beforehand (“Edit”, “Delete”, “InviteContactToMeeting” and “SendMailToContact” tasks). None of these tasks can be performed if the user does not select an existing contact previously (“Select” task), indicated by the Enabling operator (>>) connecting these tasks. “InviteContactToMeeting” and “SendMailToContact” are options that are displayed in a pop-up menu when the user right-clicks over a selected contact.

If the user pushes the “New...” button in the application to create a new contact will be

received by the the task manager will change the current task model to that presented in Figure 13, which corresponds to the screenshot of the application in Figure 10. In this screen the user should enter the required information for a new contact that is its first name, last name and the group to which the contact belongs to. It may optionally enter the second name, as indicated by the brackets around the name of the task.

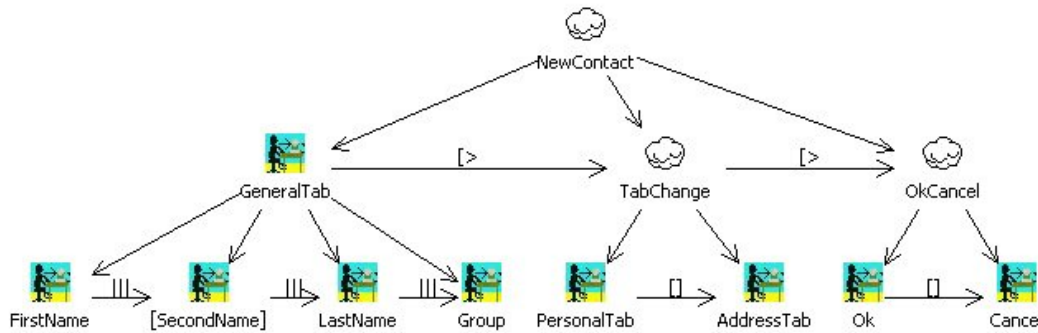


Figure 13: Task model for the general information of a new contact in Agenda 4.0

This information can be entered in the “General” tab of the screen presented to the user. But the user can enter further information by changing to the “Personal” tab or to the “Address” tab. This fact is indicated by the “TabChange” task, which is decomposed in “PersonalTab” and “AddressTab” tasks, connected by the Choice operator. Finally, the user should close the new contact dialog by accepting (“OK” task) or canceling (“Cancel” task). Note that either if the user changes to a new internal tab or if he closes the dialog, the “GeneralTab” task is disabled, as indicated by the Disabling operator ($[>]$), so the agent would not expect the user to perform its subtasks anymore.

6.4 Instantiating the framework

To use the framework presented in Section 5 with the Agenda application we have to

extend the `GUIEventListener`, `ModelManager` and `Model` classes shown in Figure 8. Figure 14 shows the extensions made to our framework's class diagram to be used in the Agenda application. We need to design a specific model manager that reflects the way in which the application changes from one task model to another. In our framework this is done by instantiating `ModelManager` class, presented in Figure 8. We create the class `AgendaModelManager`, which extends from `ModelManager`, to reflect the behavior of Agenda 4.0. This class keeps an instance of each model in the application, that are `ContactModel`, `CardModel`, `TaskModel`, `NoteModel`, `BudgetModel`, `EventModel`, `UnionModel`, `BudgetCategoriesModel`, `ContactGroupsModel`, `CardsGroupsModel`, and `EventTypesModel`. Each of these classes has its particular list of properties and various task model simulators mapping the state of the application. A set of constants is defined for the properties in each model class which are shown in Figure 14 capitalized and underlined. For example, `ContactModel` class defines the following properties: first name, second name, last name, group, address, city, state, country, zip code, occupation, birthday, phone, email, and sex. If we look at Figure 9 we see that these properties correspond to the fields shown in the table of contacts, and that is the information that the user should enter when he adds or edits a contact in the application, as we can see in Figure 10.

`ModelAgenda` is a super class for all these model classes, and is a subclass of `Model`, which represents a task model of the application. `ModelAgenda` is an abstract class that defines the common behavior of all the models in the application. It also defines a set of abstract methods that should be implemented by each particular model. For example, *`checkIfCanContinue`* method checks if the user has entered all the required information while adding a new item; *`internalTabChanged`* is use when there is a tabbed pane in the dialog for adding or editing a new item (if we look at Figure 10 we can see that the

information the user can enter for a contact is divided in three tabs: “General”. “Address” and “Personal”); *dateChanged*, *textBoxChanged*, and *comboBoxChanged* are used to detect which type of information the user entered while interacting with a date component, a text box component and a combo box component, respectively.

This set of model classes is used to keep a model of the information recently managed by the user in the application that might be useful for the process of detecting his intention and for the agent in general. The task model simulators kept by these models are used to keep track of the subtasks the user should perform to complete a task in a task model, as explained in Section 5.

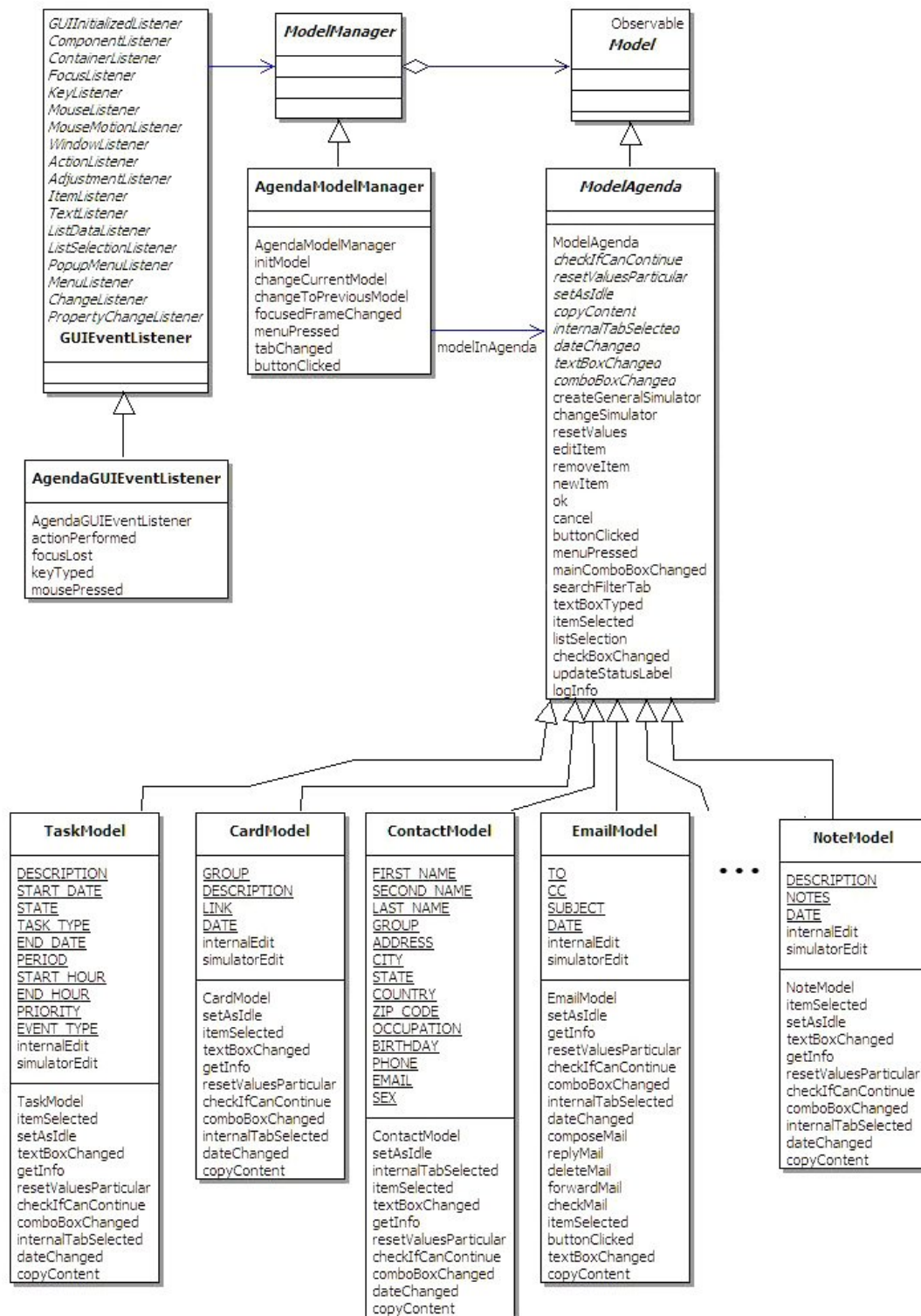


Figure 14: Extensions to our framework for the Agenda application

7 Conclusions

We presented our proposal to deal with two of the problems interface agents have to solve in order to observe the interaction of the user with the application: the observation of user actions, and the mapping of these actions to application tasks. We set as our main aim that the application's code and the agent's code should not be interwoven. In this way both units of software will be able to evolve independently one of the other.

We implemented in Java an object-oriented framework towards this goal. With this framework we tested that any interface agent effectively can get aware of the tasks performed by the user without modifying in any way the application. By using the Java Accessibility Utilities we are able to link the application with the agent in execution time. There is no linking between these two units of software in compilation time.

Our framework overcomes previous approaches in that we do not require the application to be prepared for being "observed" by the interface agent and that we do not require the availability of the source code of the application. As a result, any existent application can be enhanced with the assistance provided by an interface agent.

8 References

1. *Agents that reduce work and information overload*. **Maes, Pattie**. 7, 1994, Communications of the ACM, Vol. 37, pp. 30-40.
2. *Building plan recognition systems on arbitrary applications: the spying technique*. **Pachet, François and Giroux, Sylvain**. Montréal, Canada. : s.n., 1995. Proceedings of IJCAI-95 Workshop on The Next Generation of Plan Recognition Systems: Challenges for and Insight from Related Areas of AI.

3. *Encapsulators: A New Software Paradigm in Smalltalk-80*. **Pascoe, G.** Portland, Oregon : s.n., 1986. Proceedings of OOPSLA'86. pp. 341-346.
4. *Integrating user interface agents with conventional applications*. **Lieberman, Henry.** New York, NY, USA : ACM Press, 1998. IUI '98: Proceedings of the 3rd international conference on Intelligent user interfaces. pp. 39-46.
5. *A Task Pattern Approach to Incorporate User Deviation in Task Models*. **Basnyat, S. and Palanque, P.** Toulouse, janvier : s.n., 2005, Proceedings of the first ADVISES Young Researchers Workshop Hans H.K. Andersen and Asmatullah Nayebkheil (eds.).
6. *ConcurTaskTrees: A Diagrammatic Notation for Specifying Task Models*. **Paternò, Fabio, Mancini, C. and Meniconi, S.** 1997. Proceedings Interact'97. pp. 362-369.
7. *Task Analysis and Training Design*. **Annet, J. and Duncan, K. D.** 1967, Occupational Psychology, Vol. 41, pp. 211-221.
8. **Card, S., Moran, T. and Newell, A.** *The Psychology of Human-Computer Interaction*. Mahwah, NJ, USA : Lawrence Erlbaum Associates, Inc., 1983.
9. *GTA: Groupware task analysis - modeling complexity*. **Veer, G. van der, Lenting, B. and Bergevoet, B.** 1996, Acta Psychologica, Vol. 91, pp. 297-322.
10. *Towards a Method for Task Description : MAD*. **Scapin, D. L. and Pierret-Golbreich, C.** 1990, Work with display units.
11. *Supporting System Design by Analysing Current Task Knowledge*. **Johnson, Peter.** 1989, Task Analysis For Human-Computer Interaction, pp. 160-185.
12. *The UAN: a user-oriented representation for direct manipulation interface designs*. **Hartson, H. Rex, Siochi, Antonio C. and Hix, D.** 3, New York, NY, USA : ACM Press, 1990, ACM Trans. Inf. Syst., Vol. 8, pp. 181--203.
13. *CTTE: Support for Developing and Analyzing Task Models for Interactive System*

Design. **Mori, Giulio, Paternò, Fabio and Santoro, Carmen**. 8, 2002, IEEE Trans. Software Eng., Vol. 28, pp. 797-813.

Marcelo G. Armentano is an assistant professor in the computer science department and member of the Intelligent Agents Group at ISISTAN Research Institute at UNICEN University in Tandil, Argentina. His research interests include intelligent agents and knowledge management. He received a Systems Engineer degree at the UNICEN University in 2003 and a Master in Systems Engineer degree at the UNICEN University in 2006. Currently, he is a PhD student at the UNICEN University.

Analía A. Amandi is a professor in the computer science department and head of the Intelligent Agents Group at ISISTAN Research Institute at UNICEN University in Tandil, Argentina. Her research interests include intelligent agents and knowledge management. She received a PhD in computer science in the UFRGS, Porto Alegre, Brazil.

Figure01

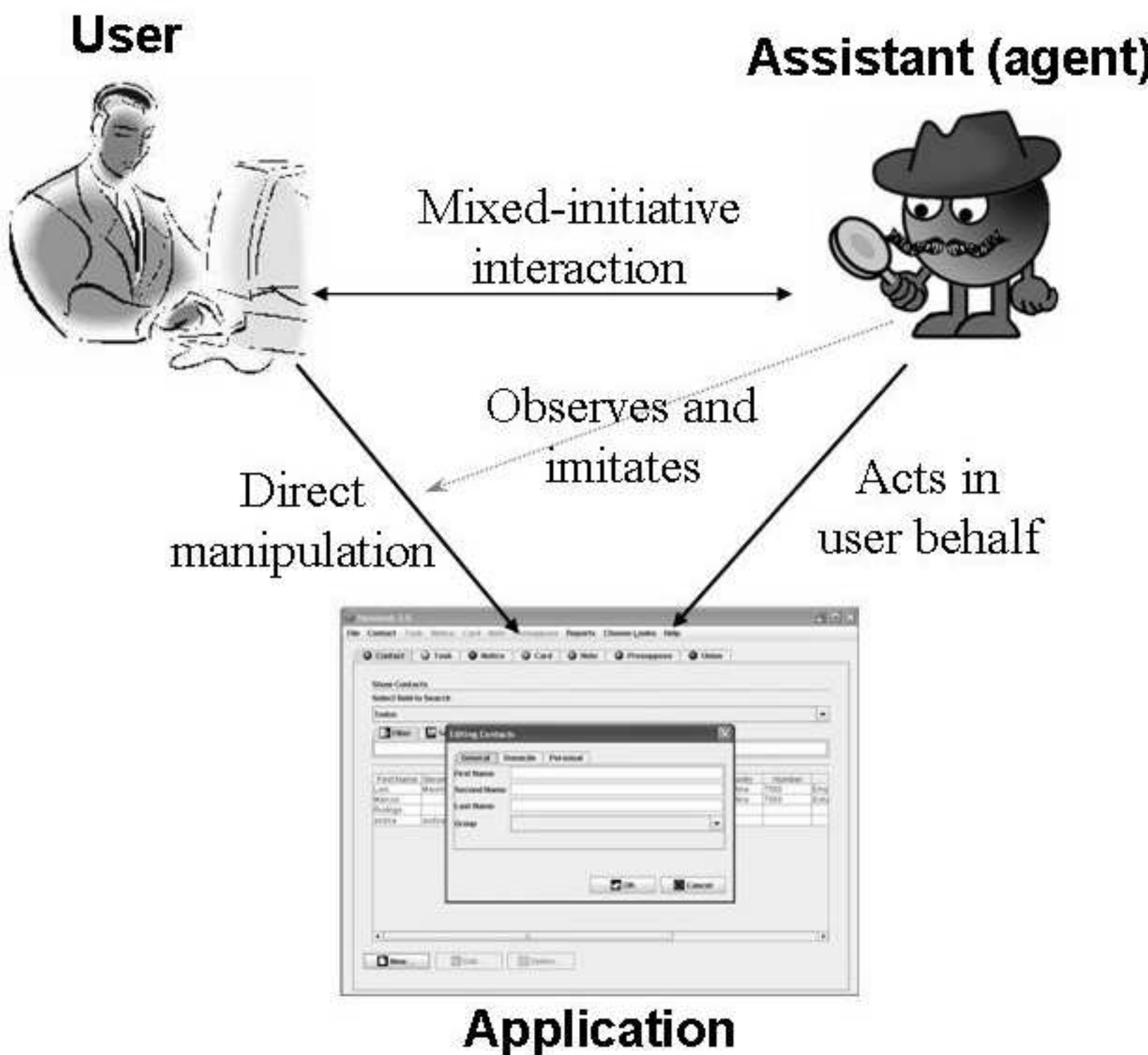


Figure02

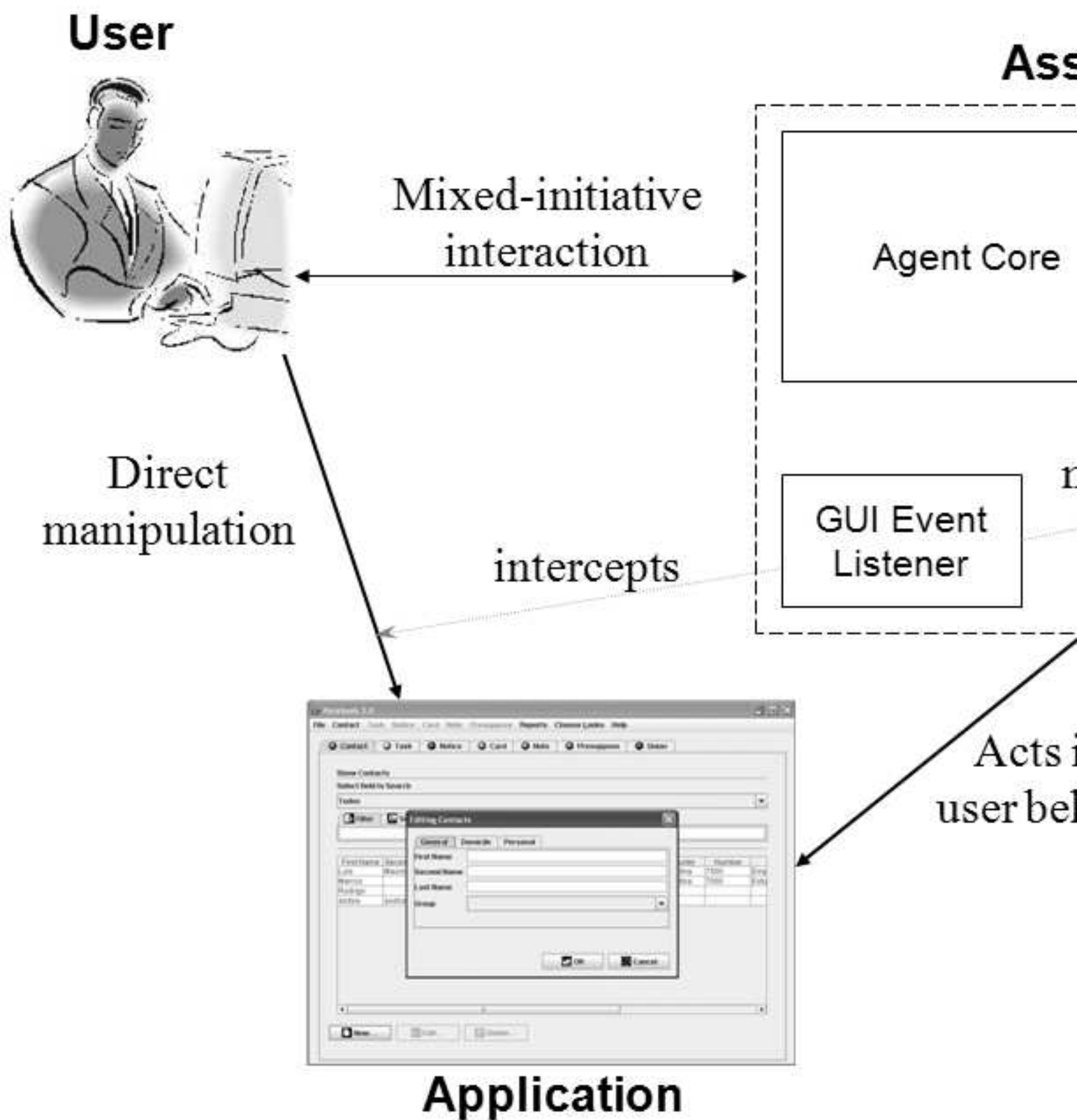


Figure03



Figure04

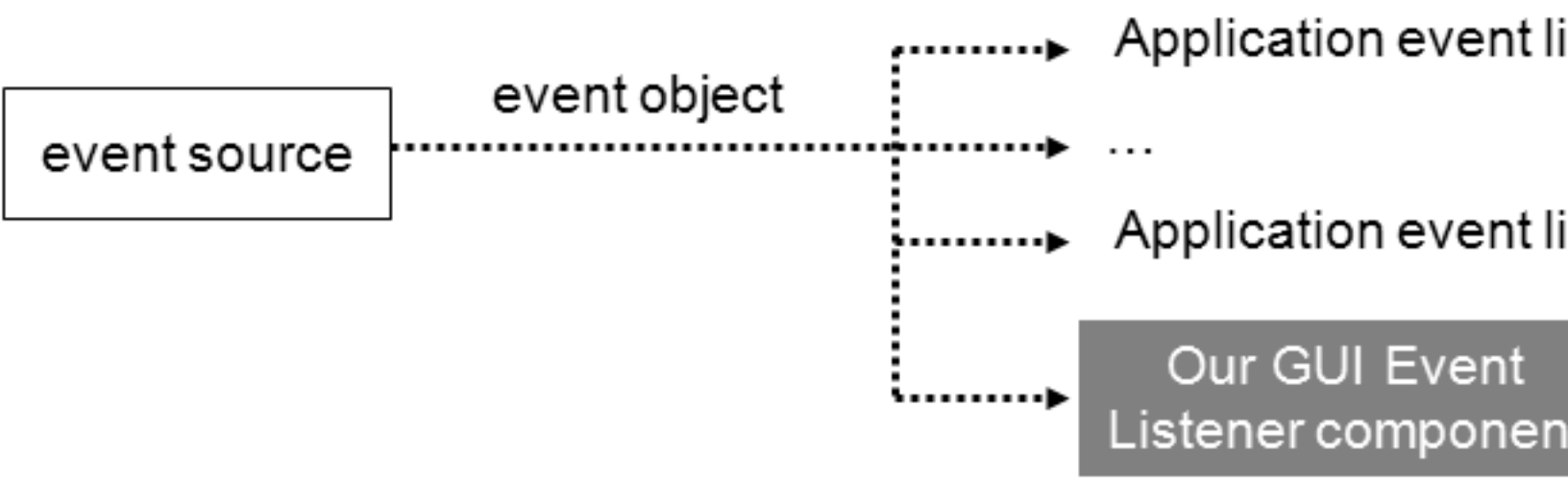


Figure05

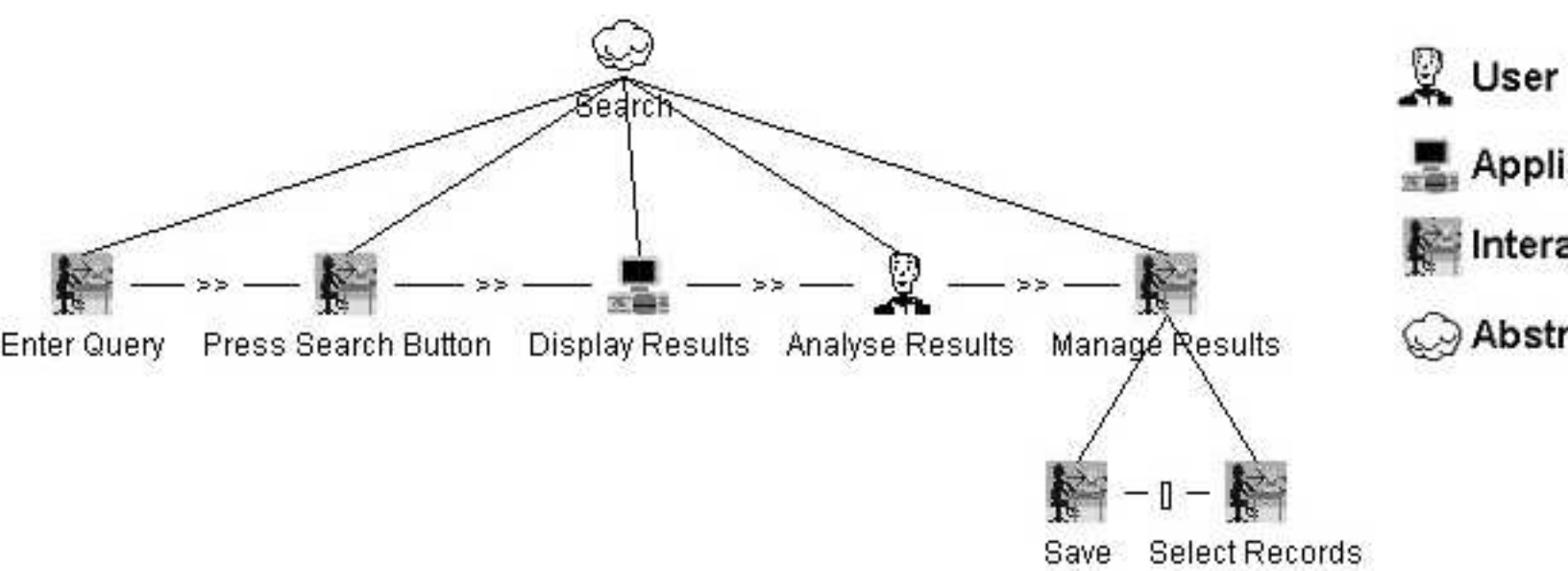


Figure06

Editing Contacts

GeneralAddressPersonal

Job

Birthday

Telephone Number

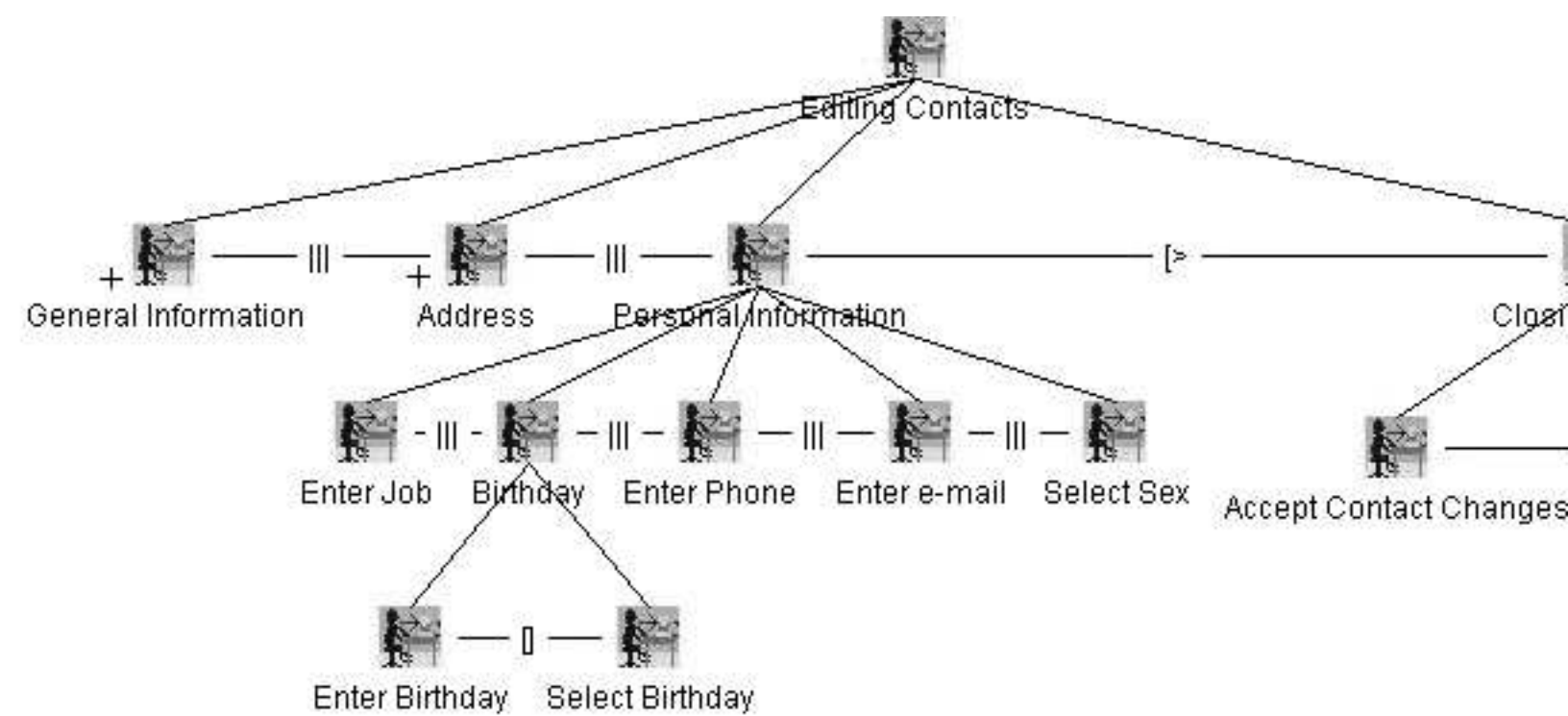
Email

Sex

OK

Cancel

Figure07



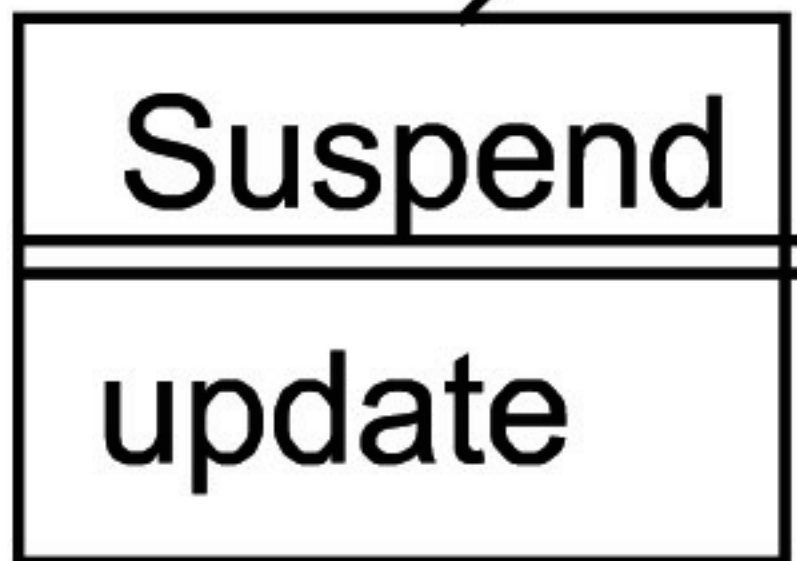


Figure09

Agenda 4.0

File Contact Task Event Card Note Budget Reports Select Looks & Feel Help

Contact Task Event Card Note Budget Union e-mail

Select field to search

All

Filter Search

First Name	Second Na...	Last Name	Group	Address	City	State	Country
Luis	Mauricio	Veyga	Family		Tres Arroyos	Buenos Air...	Argentina
Marcos		Doglioli	Family	Montevideo...	Tandil	Buenos Air...	Argentina
Luciano		Doglioli	Family	Montevideo...	Tandil	Buenos Air...	Argentina
Ezequiel		Vidal	Friends		Tandil	Buenos Air...	Argentina
Rodrigo		Patané	Friends		Buenos Air...		Argentina
Zulma		Marti	Work	Juan B Jus...	Buenos Air...		Argentina
Brandon		Routh	Friends		New York		EEUU
Bill		Gates	Work		New York		EEUU
Analia		Amandi	Work		Buenos Air...		Argentina
Luis	Sebastian	Berdun	Friends		Tandil	Buenos Air...	Argentina
Ariel		Monteserin	Friends		Tandil	Buenos Air...	Argentina
Victoria		Eyharabide	Friends		Tandil	Buenos Air...	Argentina
Francisco		Perez	Friends		Tres Arroyos		Argentina

◀ ||| ▶

New... Edit... Delete...

Figure10

Contacts

General

Address

Personal

First Name

Second Name

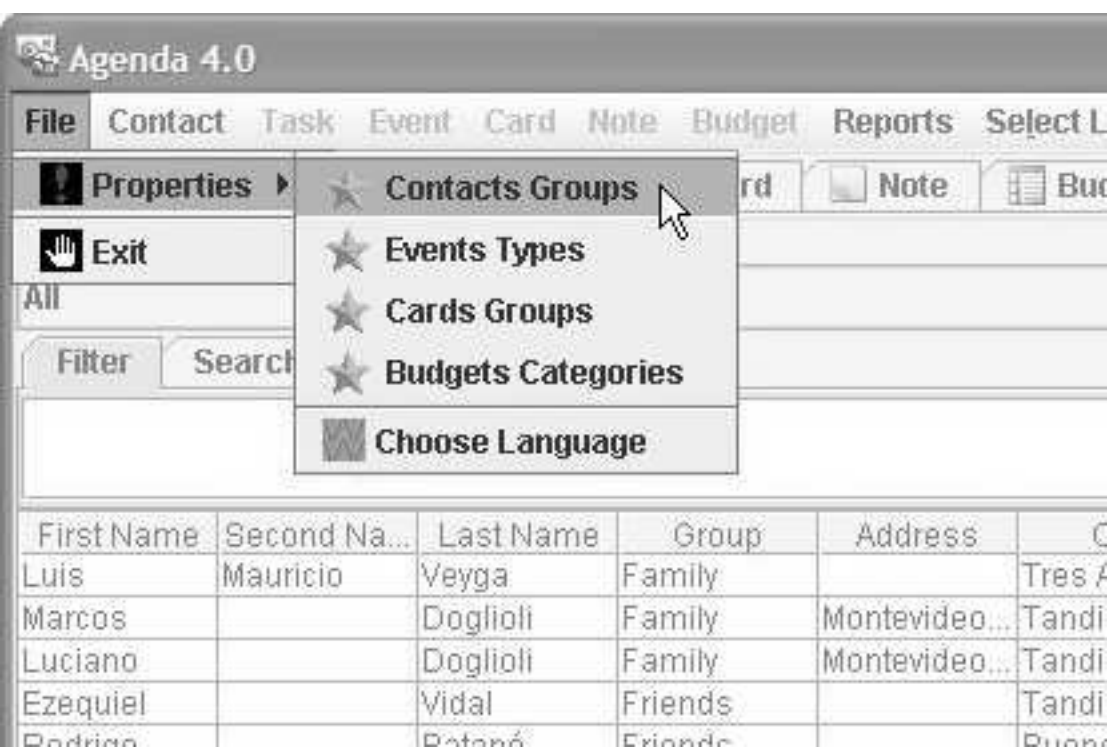
Last Name

Group

OK

Cancel

Figure11



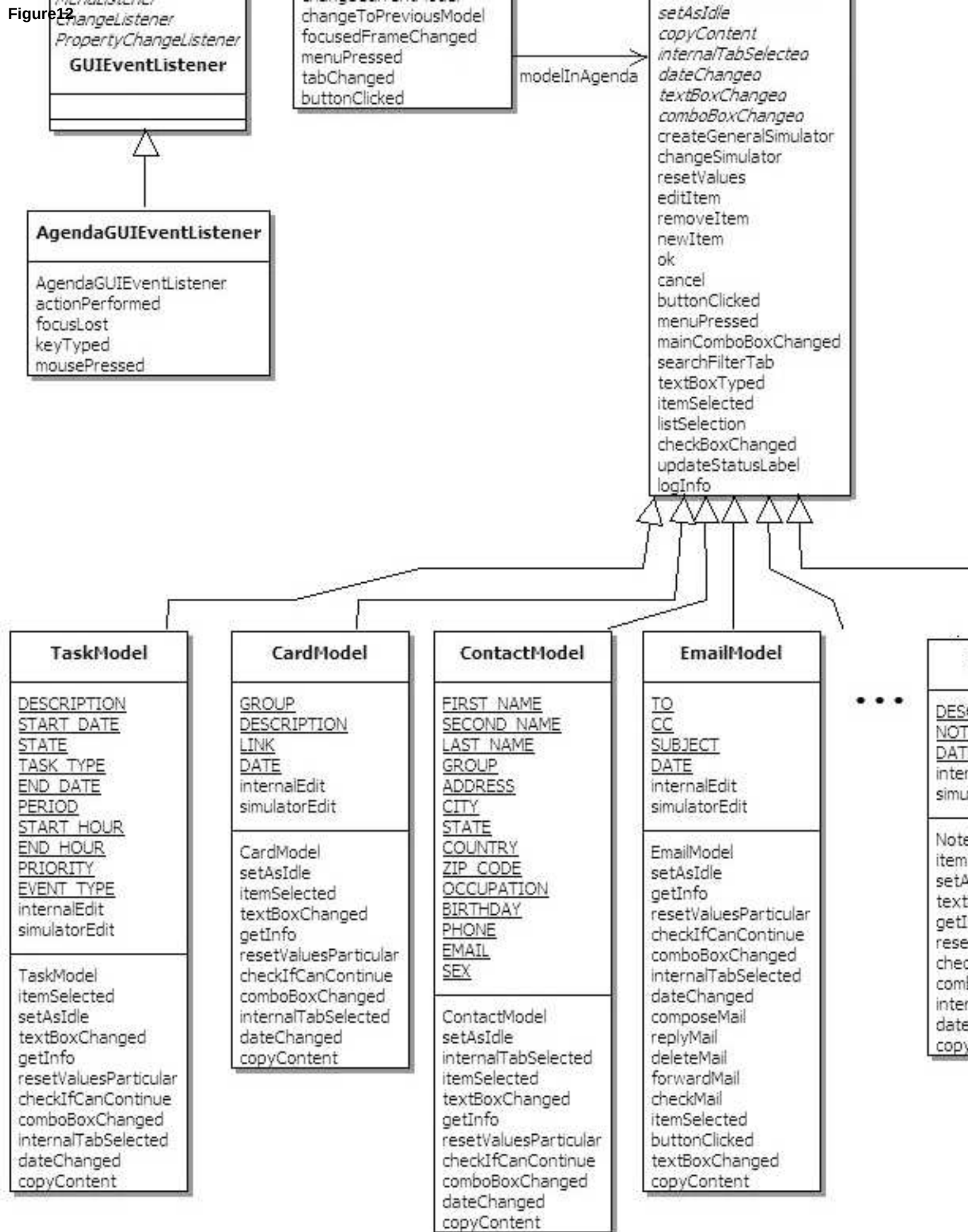


Figure13

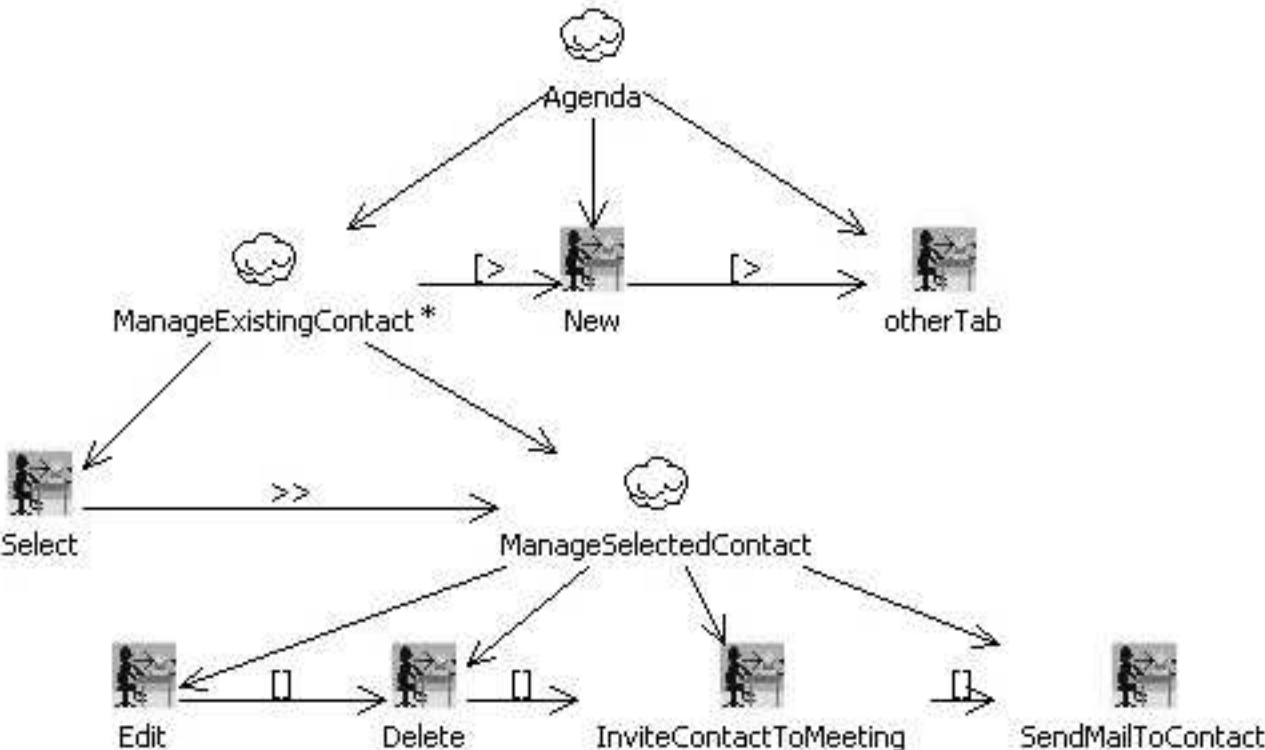


Figure14

